PUShare: Course Materials for NewUU Students in One Place

SE202 Mobile Programming · Midterm Project · Team Strangers

Abduraxmon Qutbiddinov (240426), team lead: user flow & screens
Umar Toshqulov (230070): problem research
Adxamjon Raimov (240422): presentation lead
Oyatillo Axadjonov (240446): data model

Problem statement & target users

Owned by: Umar Toshqulov.
Problem Statement
NewUU students often struggle to find course materials such as lecture slides, notes, practice exercises, and past exam papers. These resources are scattered across Telegram groups, university platforms, and shared folders, with no consistent organization.
As a result, students spend extra time searching for files, asking classmates, or checking multiple sources. Finding materials from previous semesters or specific academic years is particularly difficult.
Core problem: The lack of a centralized, structured repository for course materials makes resource discovery inefficient and time-consuming.
Target Users
Primary users: NewUU students across all majors and years of study. Our survey evidence so far comes mostly from third-year Software Engineering students, who are our first users.

Evidence: survey report.

Owned by: Umar Toshqulov.

Survey Methodology

To investigate the problem, we conducted an online survey among NewUU students between October 21 and 24, 2026.

The survey collected **19 responses** and examined how students access academic materials, the difficulties they experience, the time required to find resources, and whether these difficulties affect their assessment preparation.

Key Findings

Indicator	Results
Students who experienced difficulty finding materials	**94.7% (18/19)**
Students using Telegram for academic resources	**100% (19/19)**
Students relying on classmates for materials	**47.4% (9/19)**
Students who missed materials before quizzes or exams	**63.2% (12/19)**
Students who spent over 30 minutes finding a resource	**26.3% (5/19)**
Students experiencing difficulties daily or weekly	**57.9% (11/19)**

Analysis

The findings demonstrate that difficulties accessing academic resources are common among the surveyed students.

All 19 respondents use Telegram as a source of academic materials, while many also rely on classmates, Hero LMS, and course websites.

Furthermore, 63.2% reported that they had been unable to find required materials in time before a quiz or exam at least once.

These findings suggest a potential need for a centralized platform that organizes study materials and makes them easier to retrieve.

Conclusion

Our research provides preliminary evidence that finding academic materials is a meaningful problem for the NewUU students surveyed.

PUShare aims to address this problem by providing a centralized mobile application where students can find and share course-specific academic resources.

Research Limitations

The survey contains 19 responses, above the assignment's requested 10+. Most respondents were third-year Software Engineering students, so the results cannot be considered representative of all NewUU students.

The proposed benefits of PUShare would need to be validated through further testing.

Solution approach

Owned by: shared.

App Concept

Owned by: Adxamjon Raimov.

PUShare is a mobile app where NewUU students find and share study materials for their own major. A student signs up with their `@newuu.uz` email, and the home screen then shows only the accepted materials for their major. They can filter by academic year, study year and course, because courses and their order can differ even inside one major. Students can also upload notes, links, images and PDFs, which an admin reviews before they appear for everyone. Major, year and email are fixed after sign-up, and a student who needs a change sends a request to an admin from the Account screen. The single most important thing a user can do is find the material they need for a course in a few taps, without asking around in group chats.

Main User Flow

Owned by: Abduraxmon Qutbiddinov.
Log in with an NUU email
Login

Filter by academic year, study year, course
Home

Open a material
Material detail

Download it, reopen later without searching
Downloads

(Upload path) Share a material, which goes to pending for admin review
Upload

See whether your upload was accepted or rejected
My uploads

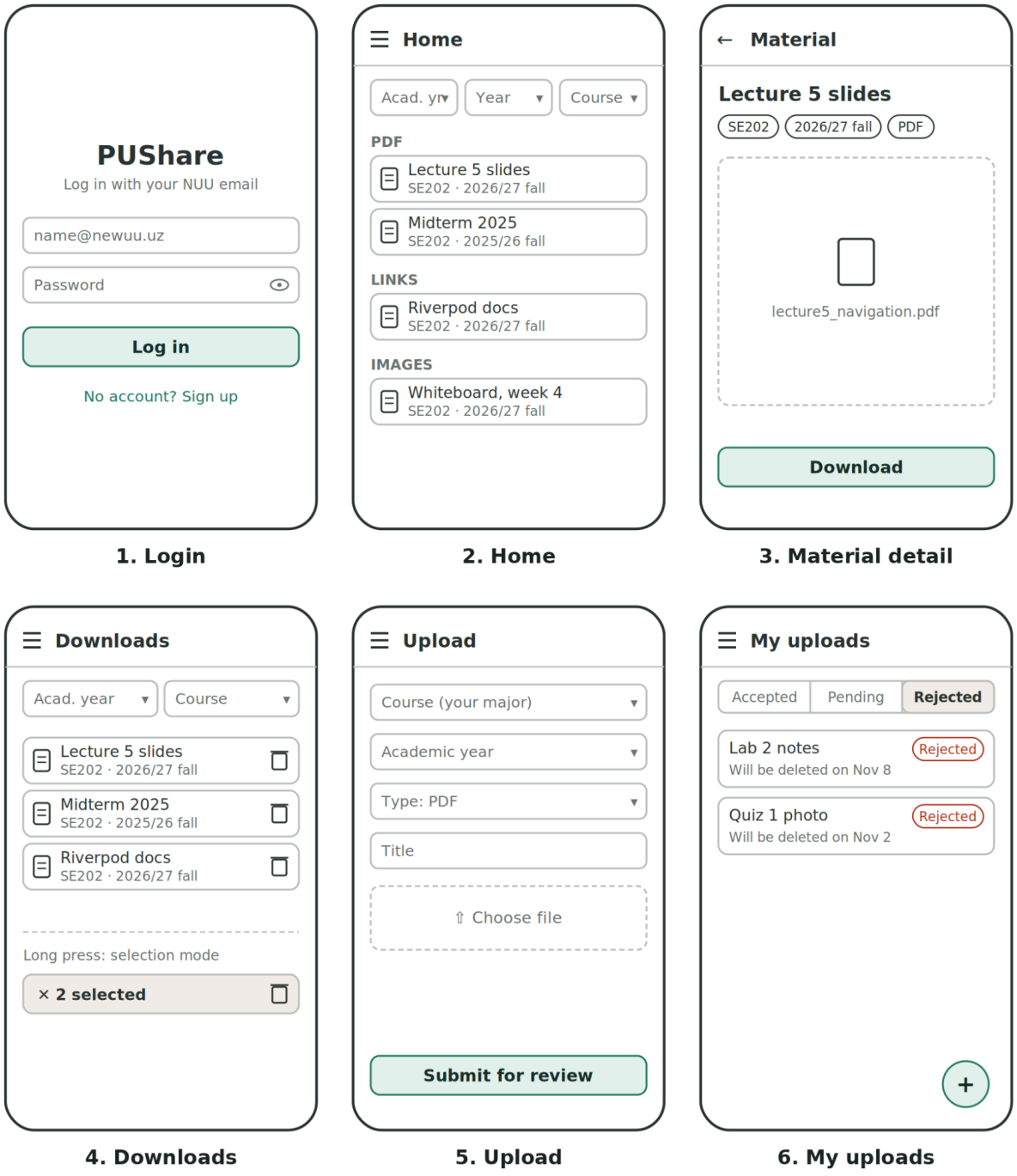
Screens & Wireframes

Screens & Wireframes
Owned by: Abduraxmon Qutbiddinov.

The app has six core screens that cover the main user flow, each with a low-fidelity wireframe. Five supporting screens reuse the same form layouts. They're described in the Flutter plan and listed below without separate wireframes.

Core screens

1. **Login.** Lets a student into the app with their @newuu.uz email.
   Wireframe: email field, password field with an eye icon, Log in button, "Sign up" link underneath.
2. **Home.** Shows the accepted materials for the student's major, so they can find a file by academic year, study year, and course.
   Wireframe: app bar with a menu icon, a row of three dropdowns (academic year, study year, course), and a list grouped under type headers (PDF, Links, Images, Text). Each card shows the title, course and academic year, and a type icon.
3. **Material detail.** Shows one material and lets the student download it.
   Wireframe: title, a row of chips (course, academic year, type), a content area (file icon and name for a PDF), and a Download button at the bottom.
4. **Downloads.** Lets the student reopen saved materials without searching again.
   Wireframe: filter bar (academic year, course) and a list of cards with a delete icon on each. Includes selection mode: the app bar shows “2 selected” with a delete action.
5. **Upload.** Lets a student share a material for a course in their major.
   Wireframe: course dropdown, academic year, type dropdown, title field, a content area (show the file picker button), and a Submit button.
6. **My uploads.** Shows the review status of everything the student has uploaded.
   Wireframe: segmented control (Accepted / Pending / Rejected), a list of cards, one rejected card with a red "Rejected" chip and the line "Will be deleted on <date>", and a + button in the corner.



Shared navigation. A side drawer with Home, Upload, My uploads, Downloads, and Account appears on all main screens.

Supporting screens (no separate wireframe)

- **Sign up:** email, major, study year, password. Email can't be changed. Major and year can only be changed through an Edit info request to an admin.
- **Confirmation code:** one 6-digit field, used for both sign-up and password change.
- **Account:** email, major, and year (read-only), plus links to Edit info, Change password, and Delete account.
- **Edit info** and **Change password:** single-purpose forms.

Flutter plan

Owned by: Oyatillo Axadjonov.

NOTE: rough sketch only.

Packages and project structure

Besides core Flutter we use two packages: `flutter_riverpod` for state and `go_router` for navigation. We have no backend yet, so the app reads from repository classes that return fake in-memory data after a short `Future.delayed`. Screens call them with `async/await`. When a real backend exists, we swap the repository implementation and the screens stay the same.

The `lib` folder is organised like this: `main.dart`, `app/` (router and theme), `models/`, `data/` (repositories), `providers/`, `screens/` (one folder per screen) and `widgets/` (shared pieces like the drawer and the material card). `main.dart` wraps the app in a `ProviderScope`, and `MyApp` returns `MaterialApp.router`.

Navigation

Route	Screen
`/login`	Login
`/signup`	Sign up
`/confirm?purpose=signup` or `changePassword`	Confirmation code (one screen, used by both flows)
`/`	Home
`/upload`	Upload
`/my-uploads`	My uploads

`/downloads`	Downloads
`/account`	Account
`/materials/:id`	Material detail
`/account/edit-info`	Request an admin for changing major/year
`/account/change-password`	Change password

The five drawer destinations (Home, Upload, My uploads, Downloads, Account) are ordinary routes. Each of these screens has its own `Scaffold` with `drawer: const AppDrawer()`, so the hamburger icon appears in its `AppBar`. `AppDrawer` is a shared stateless widget, so the drawer is defined once and reused. Drawer items close the drawer and call `context.go`, which replaces the stack. Material detail and the account sub-forms are opened with `context.push`, so the back button returns to the previous screen.

Drawer items use `go` on purpose: a drawer switches between top-level sections, so there should be no back history between them. Going back from the material detail page returns to the screen it was opened from, because Home, Downloads and My uploads open it with `push`. The tradeoff is that the previous section's local state (for example scroll position) is lost when the user switches sections. Keeping it would need `StatefulShellRoute`, which we list as out of scope.

Material detail only receives the `:id` from the path and looks the material up through a provider. This way we never pass a whole object between screens and there is one source of truth.

The confirmation screen is opened with `context.push<bool>` and closes with `context.pop(true)` on success. The caller checks the result: after sign-up it goes to Home, after a password change it goes back to Account. An auth guard in `GoRouter.redirect` sends logged-out users to `/login` and keeps logged-in users away from `/login` and `/signup`.

State management

We use Riverpod. State that more than one screen needs goes into a provider. State that only one screen needs stays in the widget.

State	Where it lives	Reason

Current user	`authProvider` (`AsyncNotifier<User?>`)	Used by the router guard, the drawer and the upload screen
All materials	`allMaterialsProvider` (`FutureProvider`)	Single source of truth
Home filters	`homeFiltersProvider` (`Notifier<MaterialFilters>`)	Must survive opening a detail page and coming back
Home list	`homeMaterialsProvider` (derived)	Watches materials, filters and user; keeps accepted materials of the user's major
One material	`materialByIdProvider` (`.family`)	Used by the detail screen
Downloads	`downloadsProvider` (`Notifier<List<DownloadRecord>>`)	Shared by Downloads and the detail screen
My uploads per status	`myUploadsProvider` (`.family` by status)	One list per tab
Form fields, controllers, dropdown values, "submitting" flag, multi-select set	Local widget state (`setState`)	Only one screen uses them and they are thrown away on exit
Is downloaded	isDownloadedProvider (.family, derived from downloadsProvider)	Decides whether the detail screen shows “Download” or “Delete from downloads”

In `build` we use `ref.watch`, in callbacks (button presses) we use `ref.read`.

Forms and validation

Every form uses `Form`, a `GlobalKey<FormState>` and `TextFormField` (or `DropdownButtonFormField` for dropdowns). On submit we call `_formKey.currentState!.validate()` first. Controllers are created in `initState` and disposed in `dispose`.

Field	Rule
Email	Not empty, ends with `@newuu.uz`
Password	At least 8 characters
Confirm password	Same as the new password field
Major, year, course, academic year, type	Must be selected
Confirmation code	Exactly 6 digits
File	Must be an allowed type
Link	Valid `http` or `https` URL
Text material	Not empty

Screens (widgets & navigation, etc.)

Login. A `Scaffold` with a padded `Column`: email field, password field (obscured, with an eye `IconButton`), a login button and a `TextButton` that goes to sign-up. Controllers are local and the button calls the login method on `authProvider`.

Sign up. A `SingleChildScrollView` around a `Form` and a `Column`, so the keyboard does not cause an overflow. It has an email field, a major dropdown, a year dropdown (1 to 4), a password field and a submit button. On submit it validates, awaits the sign-up call, pushes the confirmation screen and goes Home if the result is `true`. Major and year can only be set here. Afterwards they are read-only and only can be edited through admin by contacting them. Email is always unchangeable.

Confirmation code. A `Column` with an explanation text, one 6-digit field, a Confirm button and a Resend `TextButton`. The `purpose` query parameter decides which text is shown and which repository call is made.

Home. A `Scaffold` with the `AppDrawer`, and a `Column` containing a filter `Row` and an `Expanded` list. The `Row` has three `Expanded` dropdowns (academic year, study year, course). The materials are shown grouped by type: each type name appears as a header, followed by the cards of that type, all in one vertical scroll. To build this with a single `ListView.builder`, we first flatten the grouped data into one list of items, where an item is either a type name (`String`) or a `StudyMaterial`. The builder checks the item's type and returns a header `Text` or a `MaterialCard`. Changing a filter updates `homeFiltersProvider`, which rebuilds the list. Tapping a card calls `context.push('/materials/<id>')`. An empty result shows an empty-state widget.

Material detail. A `Scaffold` with an `AppBar` and a `ListView`: the title, a `Wrap` of `Chip`s (course, academic year, type) and the content. The content depends on the material type (`Image.network` for images, `Text` for text, link text for links, file icon and name for PDFs). The bottom button says "Download" if the material is not downloaded yet and "Delete from downloads" if it is, based on `isDownloadedProvider(id)`.

Upload. A `Form` inside a `ListView`. The fields are: course dropdown (only courses of the user's own major, which is the upload constraint), academic year, type, title, and a content field that changes with the type (URL field for links, multiline field for text, file picker button for images and PDFs). On submit we set `_submitting = true` to disable the button, await `repo.add(...)` with status `pending`, check `mounted`, then call `context.go('/my-uploads')`.

My uploads. A `Column` with a `SegmentedButton` (Accepted, Pending, Rejected) above an `Expanded(ListView.builder)` of material cards. A `FloatingActionButton` opens the upload screen. On the Rejected tab each card shows a rejected chip and the line "Will be deleted from the server on <date>". Multi-select and delete work the same way as on Downloads, through one shared selectable list widget.

Downloads. Similar view/behavior to Home. Groups its list by type, using the same grouping function as Home. Each card has a delete icon, and a long press selects several cards to delete at once.

Account. A `ListView` of `ListTile`s. Email, major and year are shown read-only. Below them are Edit info, Change password and Delete account. Edit info and Change password use `context.push`. Delete account first shows an `AlertDialog` to confirm, then calls the delete method on `authProvider`, and the router guard sends the user to `/login`.

Edit info. Major and year cannot be changed in the app, so this screen is a request form. It has a dropdown for what to change (major or study year), a second dropdown for the new value (the options depend on the first choice), and an optional multiline note for the admin. On submit it

validates, sets `_submitting = true`, awaits `repo.sendInfoChangeRequest(field, newValue, note)`, checks `mounted`, shows a `SnackBar` (“Request sent to admin”) and calls `context.pop()`. The backend is responsible for emailing the admin. Until it exists, the fake repository only simulates the delay.

Change password. A `Form` with current password, new password and confirm new password. On submit it validates, awaits the change request, pushes the confirmation screen with `push<bool>`, and goes back to Account if the result is `true`.

Shared widgets. `AppDrawer` is a `Drawer` with a `ListView` of five `ListTile`s. It highlights the current item by comparing the current route location with each item’s path.

Data model

Owned by: Oyatillo Axadjonov

We need four classes and two enums. The class is called `StudyMaterial` instead of `Material`, because `Material` is already a Flutter widget and the two names clash when `material.dart` is imported.

```
enum MaterialStatus { pending, accepted, rejected }

enum MaterialType { text, image, link, pdf }

class User {
  final String email;
  final String major;
  final int studyYear;
  final DateTime createdAt;

  const User({
    required this.email,
    required this.major,
    required this.studyYear,
    required this.createdAt,
  });
}

class Course {
  final String id;
  final String name;
  final String major;
  final int studyYear;
  final DateTime createdAt;

  const Course({
    required this.id,
    required this.name,
    required this.major,
    required this.studyYear,
    required this.createdAt,
  });
}

class StudyMaterial {
  final String id;
  final String courseId;
  final String major;
  final String academicYear;   // for example "2026/2027 fall"
  final String title;
  final MaterialType type;
  final String content;        // text body, URL, or file name
  final String? description;
  final MaterialStatus status;
  final String uploadedByUserEmail;
  final DateTime uploadedAt;
  final DateTime? reviewedAt;

  const StudyMaterial({
```

```
    required this.id,
    required this.courseId,
    required this.major,
    required this.academicYear,
    required this.title,
    required this.type,
    required this.content,
    this.description,
    required this.status,
    required this.uploadedByUserEmail,
    required this.uploadedAt,
    this.reviewedAt,
  });

  // When a rejected material is removed from the server.
  DateTime? get scheduledDeletion =>
      status == MaterialStatus.rejected && reviewedAt != null
          ? reviewedAt!.add(const Duration(days: 14))
          : null;
}

class DownloadRecord {
  final String materialId;
  final DateTime downloadedAt;
  final String localPath;

  const DownloadRecord({
    required this.materialId,
    required this.downloadedAt,
    required this.localPath,
  });
}
```

The fields map to the screens as follows:

- Sign up and Account show `User.email`, `major` and `studyYear`.
- The Home filters use `StudyMaterial.academicYear`, `Course.studyYear` and the course name (found through `courseId`).
- Lists group by `type` and show `title` and `academicYear`.
- Material detail shows `title`, `type`, `content`, the course name and `academicYear`.
- Upload fills `courseId`, `academicYear`, `type`, `title` and `content`, and sets `status` to `pending`.
- My uploads sorts by `status` and shows `scheduledDeletion` on rejected items.
- Downloads reads `DownloadRecord`, which is joined to `StudyMaterial` by `materialId`.

The filter values are kept in a small UI-only class, `MaterialFilters { String? academicYear; int? studyYear; String? courseId; }`. It is not stored anywhere.

Out of scope

Owned by: Adxamjon Raimov

The app we build for the course covers the screens, navigation, state and forms. Everything below is needed for a real product, but we haven't learned it yet, so the prototype fakes it or leaves it out.

1. **Backend and database.** Accounts, materials and courses live in fake in-memory repositories. A real server would store them and keep them in sync between users.
2. **Real authentication and email codes.** The 6-digit confirmation code is simulated. Actually sending it to an `@newuu.uz` inbox and checking it needs a server.
3. **Admin review panel.** Pending, accepted and rejected statuses exist in the app, but the admin who changes them does not. In the prototype the statuses are preset fake data.
4. **Messages to the admin.** The Edit info form lets a student request a change to their account details, but delivering that message to an admin needs a backend. In the prototype the form only validates and shows a "request sent" confirmation.
5. **File storage and real downloads.** Uploading and storing PDFs and images on a server, saving them to the phone, and cleaning up rejected files periodically needs file storage and background tasks.
6. **Push notifications.** Telling a student "your upload was accepted/rejected" would need a notification service.
7. **Keeping screen state between drawer sections.** Drawer items use `context.go`, which replaces the navigation stack, so there is no back history between sections. The tradeoff is that the previous section's local state (for example scroll position) is lost when the student switches sections. Keeping it would need `StatefulShellRoute`, which we haven't learned yet.
8. **Community voting and AI moderation.** The idea where students vote on uploads and a small AI model normalizes the course info and checks for rogue uploads is a later enhancement, not part of this version.

Who did what

Abduraxmon Qutbiddinov (team lead): main user flow, screens and wireframes; coordinated the team, made final consistency fixes across sections, assembled and submitted the document.

Umar Toshqulov: problem statement and target users; designed and distributed the survey, collected 19 responses, and analysed the results.

Adxamjon Raimov: app concept and the out-of-scope list; prepared and rehearsed the presentation.

Oyatillo Axadjonov: Flutter plan (packages, project structure, navigation, state management, forms and screen layouts) and the data model.

AI usage details

We used Claude (Anthropic) to help structure the document, improve wording, check the Flutter plan and data model for errors, draw the low-fidelity wireframes from our screen descriptions, and build the presentation slides.

Appendix: Survey evidence

Online Google Form, “How NUU Students Find Course Materials,” October 21–24, 2026, 19 responses.

Questions

1. Your year (1 / 2 / 3 / 4)
2. Your major (list of NewUU majors)
3. Where do you usually get course materials from (pick all that apply)? Telegram / Hero LMS / instructor by email / from friends or other students / paper copies / course websites
4. How often did you have trouble finding materials you needed? Daily / Weekly / Monthly / Occasionally / Never
5. Last time you looked for a specific material, how long did it take you to find it? Under 2 min / 2–10 min / 10–30 min / over 30 min / never found it

6. Have you ever missed a material before a quiz or exam because you couldn’t find it in time? Yes, once / Yes, more than once / No

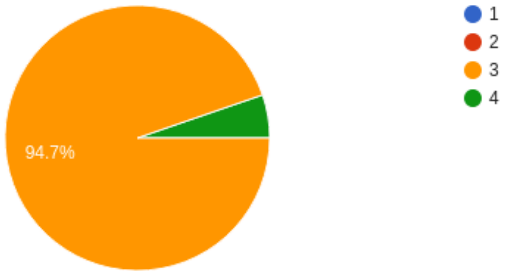
Results

Questions Responses 19 Settings

Your year

Copy chart

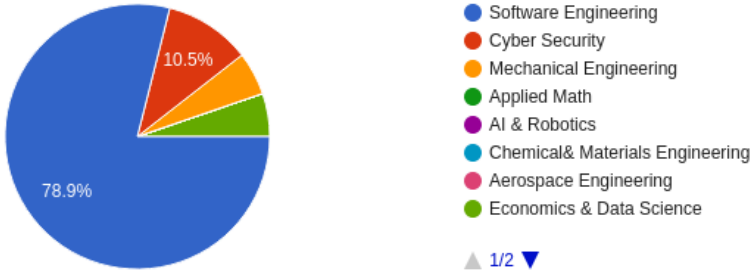
19 responses



Your major

Copy chart

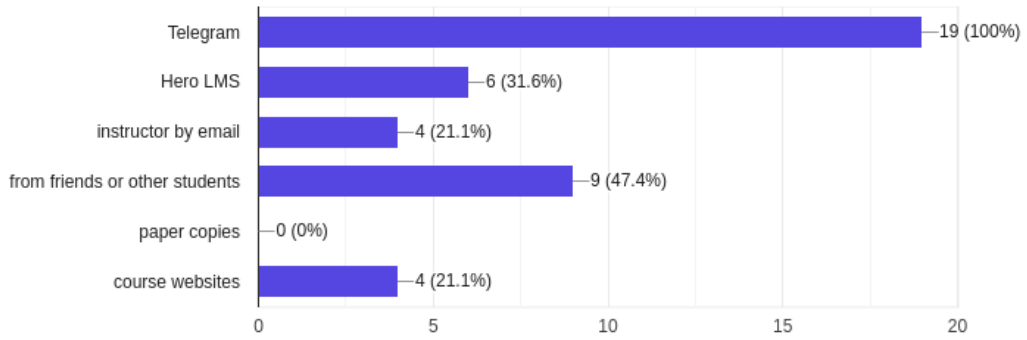
19 responses



Where do you usually get course materials from(pick all that apply)?

Copy chart

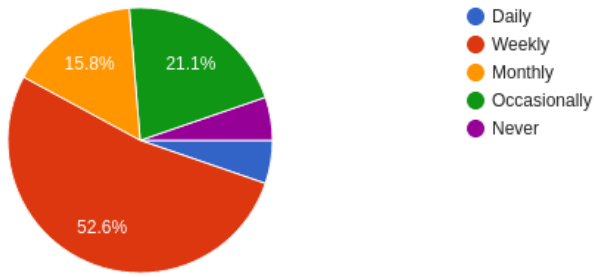
19 responses



How often did you have trouble finding materials you needed?

Copy chart

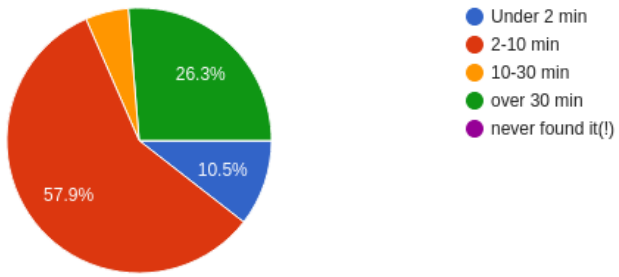
19 responses



Last time you have looked for a specific material , how long did it take you to find it?

Copy chart

19 responses



Have you ever missed a material before a quiz or exam because you couldn't find it in time?

Copy chart

19 responses

